

Practical No.1

Introduction to Excel

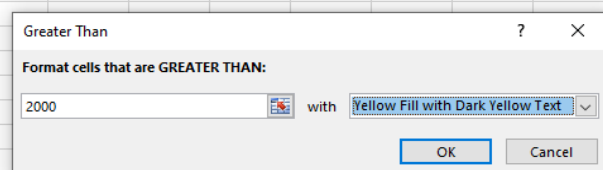
A] Perform Conditional formatting on a dataset using various criteria.

Step1: Create a Table as Given Below

Month	Sales
January	1200
February	1300
March	1600
April	1800
May	1900
June	2000
July	6500
August	4000
September	3000
October	1500
November	4500
December	1100

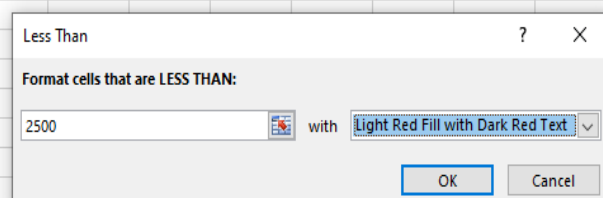
Step2: Conditional Formatting for Greater than

Month	Sales
January	1200
February	1300
March	1600
April	1800
May	1900
June	2000
July	6500
August	4000
September	3000
October	1500
November	4500
December	1100



Step3: Conditional Formatting for Less than

Month	Sales
January	1200
February	1300
March	1600
April	1800
May	1900
June	2000
July	6500
August	4000
September	3000
October	1500
November	4500
December	1100



Step4: Conditional Formatting for in Between Values

Between ? X

Format cells that are BETWEEN:

2450 and 5150 with Light Red Fill with Dark Red Text

OK Cancel

Step5: Conditional Formatting for Equal to Values

Equal To

Format cells that are EQUAL TO:

2000 with Light Red Fill

OK Cancel

Step6: Conditional Formatting for Text That Contains NA

Text That Contains ? X

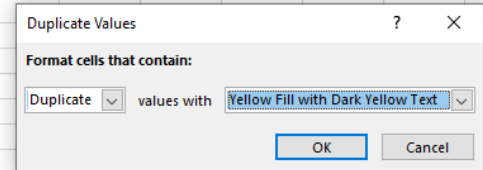
Format cells that contain the text:

na with Yellow Fill with Dark Yellow Text

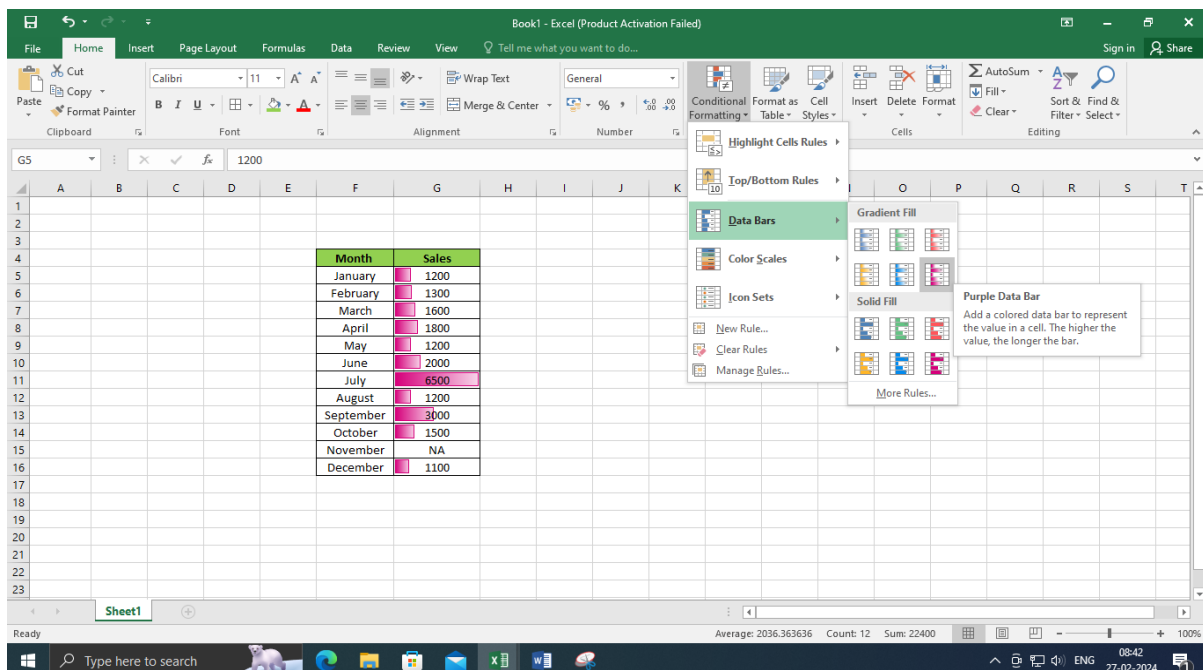
OK Cancel

Step7: Conditional Formatting for Duplicate Values

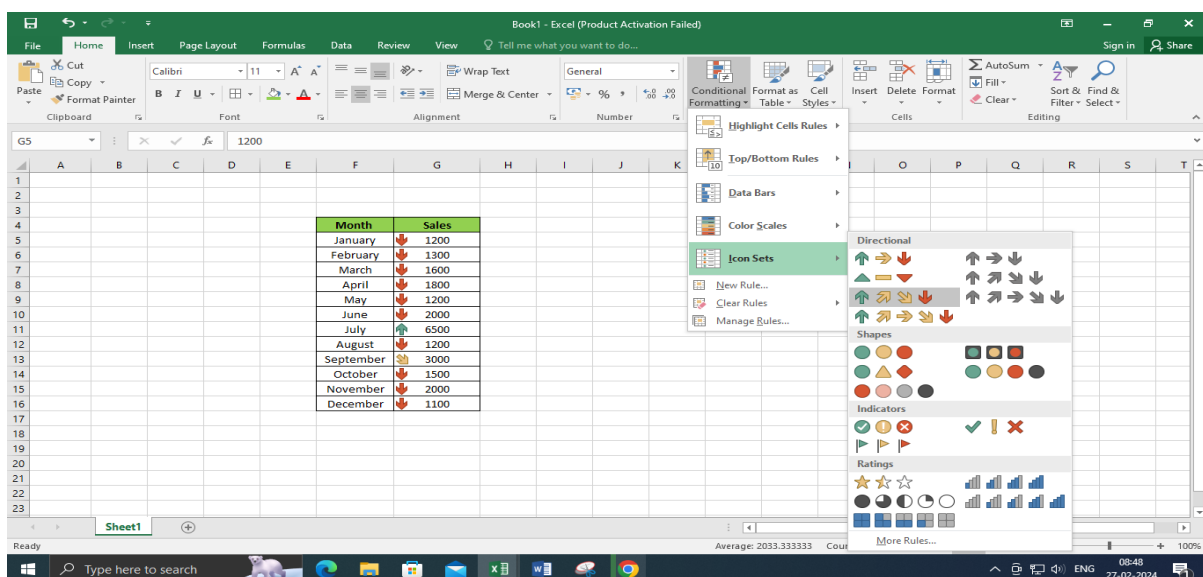
Month	Sales
January	1200
February	1300
March	1600
April	1800
May	1200
June	2000
July	6500
August	1200
September	3000
October	1500
November	NA
December	1100



Step8: Conditional Formatting for Data Bars



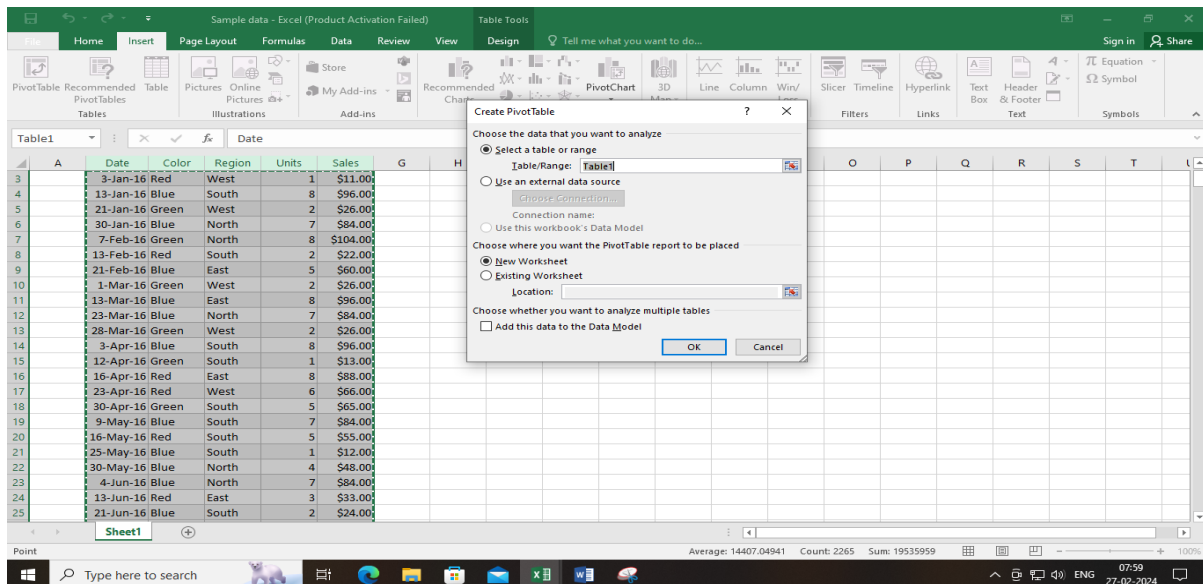
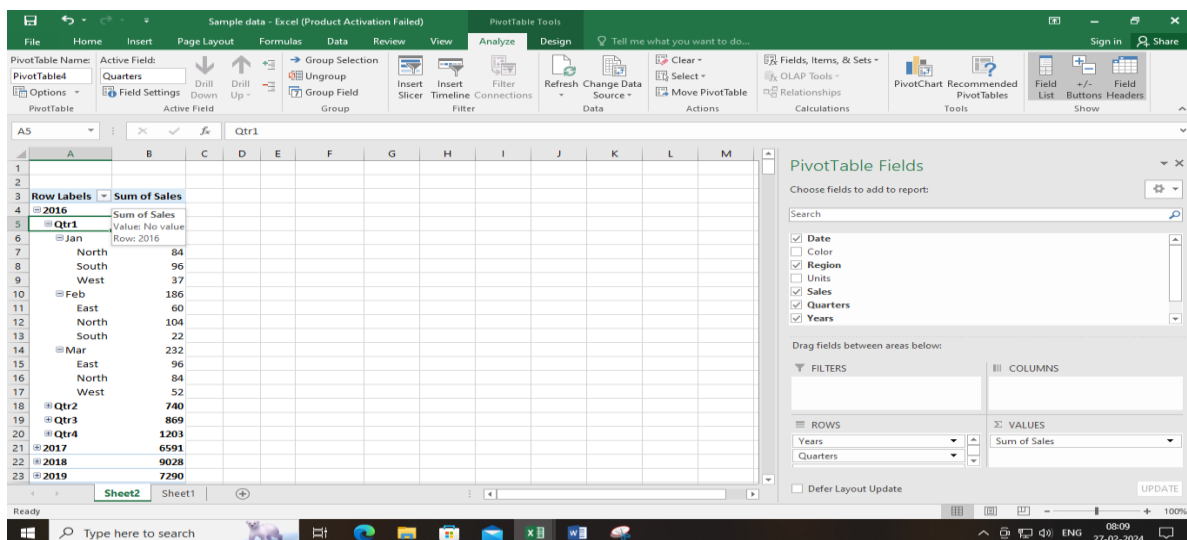
Step9: Conditional Formatting for Icons Set



B] Create A pivot table to analyse and summarize data.

Step1: Create a Sample data

A	B	C	D	E	F	G	H
	Date	Color	Region	Units	Sales		
	3-Jan-16	Red	West	1	\$11.00		
	13-Jan-16	Blue	South	8	\$96.00		
	21-Jan-16	Green	West	2	\$26.00		
	30-Jan-16	Blue	North	7	\$84.00		
	7-Feb-16	Green	North	8	\$104.00		
	13-Feb-16	Red	South	2	\$22.00		
	21-Feb-16	Blue	East	5	\$60.00		
	1-Mar-16	Green	West	2	\$26.00		
	13-Mar-16	Blue	East	8	\$96.00		
	23-Mar-16	Blue	North	7	\$84.00		
	28-Mar-16	Green	West	2	\$26.00		
	3-Apr-16	Blue	South	8	\$96.00		
	12-Apr-16	Green	South	1	\$13.00		
	16-Apr-16	Red	East	8	\$88.00		
	23-Apr-16	Red	West	6	\$66.00		
	30-Apr-16	Green	South	5	\$65.00		
	9-May-16	Blue	South	7	\$84.00		
	16-May-16	Red	South	5	\$55.00		
	25-May-16	Blue	South	1	\$12.00		
	30-May-16	Blue	North	4	\$48.00		
	4-Jun-16	Blue	North	7	\$84.00		

Step2: Insert pivot table.drag the columns in the respective fields as shown below.**Step3: Analysis and Summary of data**

C] Use VLOOKUP function to retrieve information from a different worksheet or table.

Step1: Retrieve the First Name

Function Arguments

VLOOKUP

Lookup_value: F4 = 990678

Table_array: \$A\$4:C16 = ('110608','Doe','John';253072;'Cline'...

Col_index_num: 3 = 3

Range_lookup: FALSE = FALSE

Formula result = Brad

Use VLOOKUP formulas to bring in the first and last names of employees in the Raw Data Extract section

Employee ID	Last Name	First Name
110608	Doe	John
253072	Cline	Andy
352711	Smith	John
391006	Pan	Peter
392128	Favre	Bret
549457	Elway	John
580622	Manning	Eli
602693	Vick	Micheal
611810	Woods	Tiger
612235	Jordan	Micheal
795574	Stark	Tony
830385	Williams	Prince
990678	Pitt	Brad

Employee ID	Pay	First Name	Last Name
990678	\$84,289	Brad	Pitt
830385	\$1,37,670	Prince	Williams
795574	\$1,90,024	Tony	Stark
580622	\$1,22,604	Eli	Manning
549457	\$1,11,709	John	Elway
392128	\$85,931	Bret	Favre
391006	\$1,68,114	Peter	Pan
352711	\$89,627	John	Smith
253072	\$1,49,946	Andy	Cline
612235	\$1,45,893	Micheal	Jordan
611810	\$64,757	Tiger	Woods
602693	\$71,478	Micheal	Vick
110608	\$1,31,505	John	Doe

Step2: Retrieve the Last Name

=VLOOKUP(F4,\$A\$4:\$B\$16,2,FALSE)

Function Arguments

VLOOKUP

Lookup_value: F4 = 990678

Table_array: \$A\$4:\$B\$16 = ('110608','Doe';253072;'Cline';352711...

Col_index_num: 2 = 2

Range_lookup: FALSE = FALSE

Formula result = Pitt

Use VLOOKUP formulas to bring in the first and last names of employees in the Raw Data Extract section

Employee ID	Last Name	First Name
110608	Doe	John
253072	Cline	Andy
352711	Smith	John
391006	Pan	Peter
392128	Favre	Bret
549457	Elway	John
580622	Manning	Eli
602693	Vick	Micheal
611810	Woods	Tiger
612235	Jordan	Micheal
795574	Stark	Tony
830385	Williams	Prince
990678	Pitt	Brad

Employee ID	Pay	First Name	Last Name
990678	\$84,289	Brad	Pitt
830385	\$1,37,670	Prince	Williams
795574	\$1,90,024	Tony	Stark
580622	\$1,22,604	Eli	Manning
549457	\$1,11,709	John	Elway
392128	\$85,931	Bret	Favre
391006	\$1,68,114	Peter	Pan
352711	\$89,627	John	Smith
253072	\$1,49,946	Andy	Cline
612235	\$1,45,893	Micheal	Jordan
611810	\$64,757	Tiger	Woods
602693	\$71,478	Micheal	Vick
110608	\$1,31,505	John	Doe

Step3: Retrieve the City Name

Formula Bar: `=VLOOKUP(F4,$A$24:$C$36,3,FALSE)`

Employee ID	Pay	Last Name	City
990678	\$84,289	Brad	Texas
830385	\$1,37,670	Prince	Illinois
795574	\$1,90,024	Tony	Texas
580622	\$1,22,604	Eli	Ohio
549457	\$1,11,709	John	Florida
392128	\$85,931	Bret	Illinois
391006	\$1,68,114	Peter	Illinois
352711	\$89,627	John	Florida
253072	\$1,49,946	Andy	Illinois
612235	\$1,45,893	Micheal	Florida
611810	\$64,757	Tiger	Texas
602693	\$71,478	Micheal	Florida
110608	\$1,31,505	John	Ohio

Function Arguments

VLOOKUP

Lookup_value: F4 = 990678

Table_array: \$A\$24:\$C\$36 = {110608,"Columbus","Ohio";253072,...}

Col_index_num: 3 = 3

Range_lookup: FALSE = FALSE

Formula result = Texas

[Help on this function](#) OK Cancel

Use VLOOKUP formulas to bring in the last name and city of employees in the RaData Extract sections

Step4: Retrieve the Pay Band

Formula Bar: `=VLOOKUP(F4,$A$4:$B$8,2,TRUE)`

Pay Min	Pay Band
\$25,000	Level A
\$50,000	Level B
\$75,000	Level C
\$1,00,000	Level D
\$1,50,000	Level E

Employee ID	Pay	Pay Band
990678	\$84,289	Level D
830385	\$1,37,670	Level D
795574	\$1,90,024	Level E
580622	\$1,22,604	Level D
549457	\$1,11,709	Level D
392128	\$85,931	Level C
391006	\$1,68,114	Level E
352711	\$89,627	Level C
253072	\$1,49,946	Level D
612235	\$1,45,893	Level D
611810	\$64,757	Level B
602693	\$71,478	Level B
110608	\$1,31,505	Level D

Function Arguments

VLOOKUP

Lookup_value: F4 = 84289

Table_array: \$A\$4:\$B\$8 = {25000,"Level A";50000,"Level B";75000,"Level C";100000,"Level D";150000,"Level E"}

Col_index_num: 2 = 2

Range_lookup: TRUE = TRUE

Formula result = Level C

[Help on this function](#) OK Cancel

Practical No.2

Data Frames and Basic Data Pre-processing

A] Read data from CSV and JSON files into a data frame.

```
import pandas as pd
df=pd.read_csv("D:\DS\cardata.csv")
print(df)
```

	manufacturer	model	displ	year	cyl	trans	drv	cty	hwy	fl	\
0	audi	a4	1.8	1999	4	auto(l5)	f	18	29	p	
1	audi	a4	1.8	1999	4	manual(m5)	f	21	29	p	
2	audi	a4	2.0	2008	4	manual(m6)	f	20	31	p	
3	audi	a4	2.0	2008	4	auto(av)	f	21	30	p	
4	audi	a4	2.8	1999	6	auto(l5)	f	16	26	p	
..	...	...	...	...	...	...	..	...	...	..	
229	volkswagen	passat	2.0	2008	4	auto(s6)	f	19	28	p	
230	volkswagen	passat	2.0	2008	4	manual(m6)	f	21	29	p	
231	volkswagen	passat	2.8	1999	6	auto(l5)	f	16	26	p	
232	volkswagen	passat	2.8	1999	6	manual(m5)	f	18	26	p	
233	volkswagen	passat	3.6	2008	6	auto(s6)	f	17	26	p	
	class										
0	compact										
1	compact										
2	compact										
3	compact										
4	compact										
..	...										
229	midsize										
230	midsize										
231	midsize										
232	midsize										
233	midsize										
[234 rows x 11 columns]											

B] Perform basic data pre-processing tasks such as handling missing values and outliers.

##To drop the columns

```
col=["model"]
df.drop(columns=col, axis=1)
```

	manufacturer	displ	year	cyl	trans	drv	cty	hwy	fl	class
0	audi	1.8	1999	4	auto(l5)	f	18	29	p	compact
1	audi	1.8	1999	4	manual(m5)	f	21	29	p	compact
2	audi	2.0	2008	4	manual(m6)	f	20	31	p	compact
3	audi	2.0	2008	4	auto(av)	f	21	30	p	compact
4	audi	2.8	1999	6	auto(l5)	f	16	26	p	compact
...	...	...	...	...	...	...	...	...	...	...
229	volkswagen	2.0	2008	4	auto(s6)	f	19	28	p	midsize
230	volkswagen	2.0	2008	4	manual(m6)	f	21	29	p	midsize
231	volkswagen	2.8	1999	6	auto(l5)	f	16	26	p	midsize
232	volkswagen	2.8	1999	6	manual(m5)	f	18	26	p	midsize
233	volkswagen	3.6	2008	6	auto(s6)	f	17	26	p	midsize
234 rows x 10 columns										

```
cols=["hwy","class"]
```

```
df.drop(columns=cols, axis=1)
```

	manufacturer	model	displ	year	cyl	trans	drv	cty	fl
0	audi	a4	1.8	1999	4	auto(l5)	f	18	p
1	audi	a4	1.8	1999	4	manual(m5)	f	21	p
2	audi	a4	2.0	2008	4	manual(m6)	f	20	p
3	audi	a4	2.0	2008	4	auto(av)	f	21	p
4	audi	a4	2.8	1999	6	auto(l5)	f	16	p
...	...	...	...	...	...	...	...	...	...
229	volkswagen	passat	2.0	2008	4	auto(s6)	f	19	p
230	volkswagen	passat	2.0	2008	4	manual(m6)	f	21	p
231	volkswagen	passat	2.8	1999	6	auto(l5)	f	16	p
232	volkswagen	passat	2.8	1999	6	manual(m5)	f	18	p
233	volkswagen	passat	3.6	2008	6	auto(s6)	f	17	p

234 rows x 9 columns

```
##To rename the columns
```

```
new_column_names = {'model': 'car_model', 'trans': 'ts'}
```

```
df = df.rename(columns=new_column_names)
```

```
print(df)
```

	manufacturer	car_model	displ	year	cyl	ts	drv	cty	hwy	fl
0	audi	a4	1.8	1999	4	auto(l5)	f	18	29	p
1	audi	a4	1.8	1999	4	manual(m5)	f	21	29	p
2	audi	a4	2.0	2008	4	manual(m6)	f	20	31	p
3	audi	a4	2.0	2008	4	auto(av)	f	21	30	p
4	audi	a4	2.8	1999	6	auto(l5)	f	16	26	p
...	...	...	...	...	...	...	...	...	...	...
229	volkswagen	passat	2.0	2008	4	auto(s6)	f	19	28	p
230	volkswagen	passat	2.0	2008	4	manual(m6)	f	21	29	p
231	volkswagen	passat	2.8	1999	6	auto(l5)	f	16	26	p
232	volkswagen	passat	2.8	1999	6	manual(m5)	f	18	26	p
233	volkswagen	passat	3.6	2008	6	auto(s6)	f	17	26	p

class	
0	compact
1	compact
2	compact
3	compact
4	compact
...	...
229	midsize
230	midsize
231	midsize
232	midsize
233	midsize

[234 rows x 11 columns]

```
##check for empty rows
```

```
empty_rows = df.isnull().all(axis=1)
```

```
# Print empty rows
```

```
print(empty_rows)
```

```
num_empty_rows = empty_rows.sum()
```

```
print("Number of empty rows:", num_empty_rows)
```

```
0      False
1      False
2      False
3      False
4      False
...
229    False
230    False
231    False
232    False
233    False
Length: 234, dtype: bool
Number of empty rows: 0
```


Check for missing values in each row

```
missing_values_in_row = df.isnull().any(axis=1)
```

Print rows with missing values

```
print(df[missing_values_in_row])
```

```
num_missing_values_in_row = missing_values_in_row.sum()
```

```
print("Number of rows with missing values:", num_missing_values_in_row)
```

```
Empty DataFrame
```

```
Columns: [manufacturer, car_model, displ, year, cyl, ts, drv, cty, hwy, fl, class]
```

```
Index: []
```

```
Number of rows with missing values: 0
```

Drop rows with missing values

```
df_cleaned = df.dropna()
```

Print the cleaned DataFrame

```
print(df_cleaned)
```

```

0      manufacturer car_model  displ  year  cyl      ts  drv  cty  hwy  fl
1      audi          a4      1.8  1999    4    auto(l5)  f   18   29   p
2      audi          a4      1.8  1999    4    manual(m5)  f   21   29   p
3      audi          a4      2.0  2008    4    manual(m6)  f   20   31   p
4      audi          a4      2.0  2008    4    auto(av)  f   21   30   p
..      ...          ...      ...  ...  ...      ...  ..   ...   ..
229    volkswagen    passat    2.0  2008    4    auto(s6)  f   19   28   p
230    volkswagen    passat    2.0  2008    4    manual(m6)  f   21   29   p
231    volkswagen    passat    2.8  1999    6    auto(l5)  f   16   26   p
232    volkswagen    passat    2.8  1999    6    manual(m5)  f   18   26   p
233    volkswagen    passat    3.6  2008    6    auto(s6)  f   17   26   p

      class
0    compact
1    compact
2    compact
3    compact
4    compact
..      ...
229  midsize
230  midsize
231  midsize
232  midsize
233  midsize

[234 rows x 11 columns]
```

C) Manipulate and transform data using functions like filtering, sorting, and grouping

#To perform sorting, filtering and grouping

```
df2=pd.read_csv("d:\\ds\\biostats.csv")
```

```
print(df2)
```

```

0      Name  Sex  Age  Height  Weighth
1      Alex   M   41     74     170
2      Bert   M   42     68     166
3      Carl   M   32     70     155
4      Dave   M   39     72     167
5      Elly   F   30     66     124
6      Fran   F   33     66     115
7      Gwen   F   26     64     121
8      Hank   M   30     71     158
9      Ivan   M   53     72     175
10     Jake   M   32     69     143
11     Kate   F   47     69     139
12     Luke   M   34     72     163
13     Myra   F   23     62     98
14     Neil   M   36     75     160
15     Omar   M   38     70     145
16     Page   F   31     67     135
17     Quin   M   29     71     176
18     Ruth   F   28     65     131
```

Sort the DataFrame by a single column

```
sorted_df = df2.sort_values(by='Name')
print(sorted_df)
```

	Name	Sex	Age	Height	Weighth
0	Alex	M	41	74	170
1	Bert	M	42	68	166
2	Carl	M	32	70	155
3	Dave	M	39	72	167
4	Elly	F	30	66	124
5	Fran	F	33	66	115
6	Gwen	F	26	64	121
7	Hank	M	30	71	158
8	Ivan	M	53	72	175
9	Jake	M	32	69	143
10	Kate	F	47	69	139
11	Luke	M	34	72	163
12	Myra	F	23	62	98
13	Neil	M	36	75	160
14	Omar	M	38	70	145
15	Page	F	31	67	135
16	Quin	M	29	71	176
17	Ruth	F	28	65	131

Sort the DataFrame by a single column in descending order

```
sorted_df = df2.sort_values(by='Name',ascending=False)
print(sorted_df)
```

	Name	Sex	Age	Height	Weighth
17	Ruth	F	28	65	131
16	Quin	M	29	71	176
15	Page	F	31	67	135
14	Omar	M	38	70	145
13	Neil	M	36	75	160
12	Myra	F	23	62	98
11	Luke	M	34	72	163
10	Kate	F	47	69	139
9	Jake	M	32	69	143
8	Ivan	M	53	72	175
7	Hank	M	30	71	158
6	Gwen	F	26	64	121
5	Fran	F	33	66	115
4	Elly	F	30	66	124
3	Dave	M	39	72	167
2	Carl	M	32	70	155
1	Bert	M	42	68	166
0	Alex	M	41	74	170

Sort the DataFrame by multiple columns

```
sorted_df = df2.sort_values(by=['Gender', 'Age'])
print(sorted_df)
```

	Name	Sex	Age	Height	Weighth
12	Myra	F	23	62	98
6	Gwen	F	26	64	121
17	Ruth	F	28	65	131
4	Elly	F	30	66	124
15	Page	F	31	67	135
5	Fran	F	33	66	115
10	Kate	F	47	69	139
16	Quin	M	29	71	176
7	Hank	M	30	71	158
2	Carl	M	32	70	155
9	Jake	M	32	69	143
11	Luke	M	34	72	163
13	Neil	M	36	75	160
14	Omar	M	38	70	145
3	Dave	M	39	72	167
0	Alex	M	41	74	170
1	Bert	M	42	68	166
8	Ivan	M	53	72	175

Filter records

```
filtered_df = df2[df2['Height'] > 30]
print(filtered_df)
```

	Name	Sex	Age	Height	Weigth
0	Alex	M	41	74	170
1	Bert	M	42	68	166
2	Carl	M	32	70	155
3	Dave	M	39	72	167
4	Elly	F	30	66	124
5	Fran	F	33	66	115
6	Gwen	F	26	64	121
7	Hank	M	30	71	158
8	Ivan	M	53	72	175
9	Jake	M	32	69	143
10	Kate	F	47	69	139
11	Luke	M	34	72	163
12	Myra	F	23	62	98
13	Neil	M	36	75	160
14	Omar	M	38	70	145
15	Page	F	31	67	135
16	Quin	M	29	71	176
17	Ruth	F	28	65	131

```
filtered_df = df2[(df2['Age'] > 10) & (df2['Height'] < 70)]
print(filtered_df)
```

	Name	Sex	Age	Height	Weigth
1	Bert	M	42	68	166
4	Elly	F	30	66	124
5	Fran	F	33	66	115
6	Gwen	F	26	64	121
9	Jake	M	32	69	143
10	Kate	F	47	69	139
12	Myra	F	23	62	98
15	Page	F	31	67	135
17	Ruth	F	28	65	131

Grouping by a single column and performing aggregation

```
grouped_data = df2.groupby('Sex')
print(grouped_data)
grouped_data.count()
```

```
<pandas.core.groupby.generic.DataFrameGroupBy object at 0x00000284D9FCCC70>
```

	Name	Age	Height	Weigth
Sex				
F	7	7	7	7
M	11	11	11	11

Practical No.3

Feature Scaling and Dummification

A] Apply feature-scaling techniques like standardization and normalization to numerical features.

```
import numpy as np
import pandas as pd
from sklearn import preprocessing
import matplotlib.pyplot as plt
%matplotlib inline
data=pd.read_csv('D:\Rituja TYCS\Data_for_Missing_Values.csv')
data.head(15)
```

	Country	Age	Salary	Purchased
0	France	44.0	72000.0	No
1	Spain	27.0	48000.0	Yes
2	Germany	30.0	54000.0	No
3	Spain	38.0	61000.0	No
4	Germany	40.0	NaN	Yes
5	France	35.0	58000.0	Yes
6	Spain	NaN	52000.0	No
7	France	48.0	79000.0	Yes
8	Germany	50.0	83000.0	No
9	France	37.0	67000.0	Yes

```
data=data.dropna()
data
```

	Country	Age	Salary	Purchased
0	France	44.0	72000.0	No
1	Spain	27.0	48000.0	Yes
2	Germany	30.0	54000.0	No
3	Spain	38.0	61000.0	No
5	France	35.0	58000.0	Yes
7	France	48.0	79000.0	Yes
8	Germany	50.0	83000.0	No
9	France	37.0	67000.0	Yes

#calculating mean,median before feature scaling

#data.mean()

```
data.select_dtypes(include=['number']).mean()
```

```
data.mean()
```

```
Age          38.625
Salary       65250.000
dtype: float64
```

```
data.median()
```

```
Age          37.5  
Salary      64000.0  
dtype: float64
```

#slicing the feature age and salary

```
df=data.iloc[:,1:3].values  
df
```

```
array([[4.4e+01, 7.2e+04],  
       [2.7e+01, 4.8e+04],  
       [3.0e+01, 5.4e+04],  
       [3.8e+01, 6.1e+04],  
       [3.5e+01, 5.8e+04],  
       [4.8e+01, 7.9e+04],  
       [5.0e+01, 8.3e+04],  
       [3.7e+01, 6.7e+04]])
```

#MinMaxScaler or Normalization

```
min_max_scaler=preprocessing.MinMaxScaler(feature_range=(0,1))  
df_after_minmax=min_max_scaler.fit_transform(df)  
df_after_minmax
```

```
array([[0.73913043, 0.68571429],  
       [0.        , 0.        ],  
       [0.13043478, 0.17142857],  
       [0.47826087, 0.37142857],  
       [0.34782609, 0.28571429],  
       [0.91304348, 0.88571429],  
       [1.        , 1.        ],  
       [0.43478261, 0.54285714]])
```

#Standarization

```
ss=preprocessing.StandardScaler()  
df_after_ss=ss.fit_transform(df)  
df_after_ss
```

```
array([[ 0.69985807,  0.58989097],  
       [-1.51364653, -1.50749915],  
       [-1.12302807, -0.98315162],  
       [-0.08137885, -0.37141284],  
       [-0.47199731, -0.6335866 ],  
       [ 1.22068269,  1.20162976],  
       [ 1.48109499,  1.55119478],  
       [-0.211585   ,  0.1529347 ]])
```

#calculating mean,median after feature scaling

df_after_minmax.mean()

0.4991459627329192

Generate

+ Code

+ Markd

B] Perform feature dummification to convert categorical variables into numerical representations.

df= pd.read_csv("D:\\DS\\biostatsD.csv")

df

	Name	Gender	Age	Height	Weigth
0	Alex	M	41	74	170
1	Bert	M	42	68	166
2	Carl	M	32	70	155
3	Dave	M	39	72	167
4	Elly	F	30	66	124
5	Fran	F	33	66	115
6	Gwen	F	26	64	121
7	Hank	M	30	71	158
8	Ivan	M	53	72	175
9	Jake	M	32	69	143
10	Kate	F	47	69	139
11	Luke	M	34	72	163
12	Myra	F	23	62	98
13	Neil	M	36	75	160
14	Omar	M	38	70	145
15	Page	F	31	67	135
16	Quin	M	29	71	176
17	Ruth	F	28	65	131

df_dummies = pd.get_dummies(df, columns=['Gender'],drop_first=True)

Print the resulting DataFrame with dummified columns

print(df_dummies)

	Name	Age	Height	Weigth	Gender_F	Gender_M
0	Alex	41	74	170	0	1
1	Bert	42	68	166	0	1
2	Carl	32	70	155	0	1
3	Dave	39	72	167	0	1
4	Elly	30	66	124	1	0
5	Fran	33	66	115	1	0
6	Gwen	26	64	121	1	0
7	Hank	30	71	158	0	1
8	Ivan	53	72	175	0	1
9	Jake	32	69	143	0	1
10	Kate	47	69	139	1	0
11	Luke	34	72	163	0	1
12	Myra	23	62	98	1	0
13	Neil	36	75	160	0	1
14	Omar	38	70	145	0	1
15	Page	31	67	135	1	0
16	Quin	29	71	176	0	1
17	Ruth	28	65	131	1	0

Practical No.4**Hypothesis Testing**

A] Formulate null and alternative hypotheses for a given problem.

```
import numpy as np
from scipy.stats import ttest_1samp
# Sample data: Delivery times (in minutes)
delivery_times = [28, 32, 30, 35, 29, 27, 31, 34, 33, 29]
# Hypothesized population mean
hypothesized_mean = 30
# Perform one-sample t-test
t_stat, p_value = ttest_1samp(delivery_times, hypothesized_mean)
# Print results
print("T-statistic:", t_stat)
print("P-value:", p_value)
# Significance level
alpha = 0.05
# Interpret results
print("\nHypotheses:")
print("Null Hypothesis (H0): The mean delivery time is 30 minutes.")
print("Alternate Hypothesis (H1): The mean delivery time is not 30 minutes.")
if p_value < alpha:
    print(f"\nResult: Reject the null hypothesis")
else:
    print(f"Accept: Alternate Hypothesis (H1): The mean delivery time is not 30minutes.")
```

```
T-statistic: 0.9516619028617739
P-value: 0.3661247851388799

Hypotheses:
Null Hypothesis (H0): The mean delivery time is 30 minutes.
Alternate Hypothesis (H1): The mean delivery time is not 30 minutes.

Result: Fail to reject the null hypothesis (p = 0.3661 ≥ 0.05).
Conclusion: There is no significant evidence that the mean delivery time differs from 30 minutes.
```

B] Interpret the results and draw conclusions based on the test outcomes

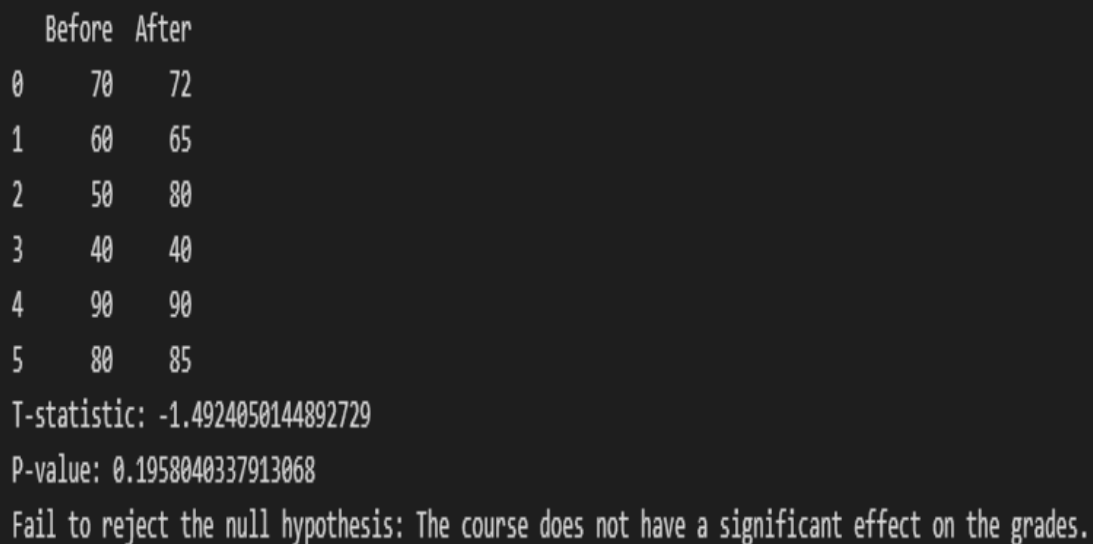
```
import pandas as pd
from scipy import stats

# Load the CSV file into a DataFrame
df = pd.read_csv("D:\\DS\\grades.csv")
```

```
print(df)
# Perform paired t-test
t_statistic, p_value = stats.ttest_rel(df['Before'], df['After'])

# Print the results
print("T-statistic:", t_statistic)
print("P-value:", p_value)

# Interpret the results
alpha = 0.05
if p_value < alpha:
    print("Accept Null Hypothesis : The course has a significant effect on the grades.")
else:
    print("Accept Alternate hypothesis: The course does not have a significant effect on the grades.")
```



	Before	After
0	70	72
1	60	65
2	50	80
3	40	40
4	90	90
5	80	85

T-statistic: -1.4924050144892729
P-value: 0.1958040337913068
Fail to reject the null hypothesis: The course does not have a significant effect on the grades.

Practical No.5**ANOVA (Analysis of Variance)**

A] Perform one-way ANOVA to compare means across multiple groups

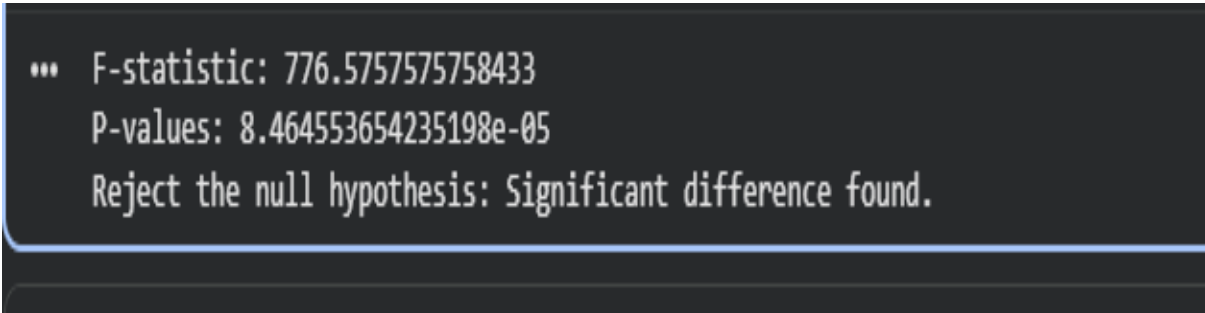
```
import pandas as pd
from scipy.stats import f_oneway

# 1. Read the file
# Note: If running on Colab, change this path to just "ANOVA.csv"
df = pd.read_csv("D:\Rituja\ANOVA.csv")

# 2. Filter the scores for each group manually
scores_A = df[df['Group'] == 'A']['Score']
scores_B = df[df['Group'] == 'B']['Score']
scores_C = df[df['Group'] == 'C']['Score']

# 3. Perform Anova
# syntax fix: added comma between f_statistic and p_value
f_statistic, p_value = f_oneway(scores_A, scores_B, scores_C)
print("F-statistic:", f_statistic)
print("P-values:", p_value)

# 4. Decision Logic
alpha = 0.05
if p_value < alpha:
    print("Reject the null hypothesis: Significant difference found.")
else:
    print("Failed to reject null hypothesis: No significant difference.")
```



```
... F-statistic: 776.5757575758433
P-values: 8.464553654235198e-05
Reject the null hypothesis: Significant difference found.
```

B] Conduct post-hoc tests to identify significant differences between group mean

```
import pandas as pd
from scipy import stats

# Load the dataset
df = pd.read_csv("D:\Rituja\mtcars.csv")
```

```

# Check first few rows
print(df.head())

# Separate mpg values based on number of cylinders
mpg_4 = df[df['cyl'] == 4]['mpg']
mpg_6 = df[df['cyl'] == 6]['mpg']
mpg_8 = df[df['cyl'] == 8]['mpg']

# Perform One-Way ANOVA
f_stat, p_value = stats.f_oneway(mpg_4, mpg_6, mpg_8)

# Print results
print("F-statistic:", f_stat)
print("P-value:", p_value)

# Decision
alpha = 0.05
if p_value < alpha:
    print("Mean mpg differs across cylinder groups")
else:
    print("No significant difference in mean mpg")

```

```

...
      model  mpg  cyl  disp  hp  drat    wt    qsec  vs  am  gear  \
0    Mazda RX4  21.0   6  160.0  110  3.90  2.620  16.46   0   1    4
1    Mazda RX4 Wag  21.0   6  160.0  110  3.90  2.875  17.02   0   1    4
2    Datsun 710  22.8   4  108.0   93  3.85  2.320  18.61   1   1    4
3    Hornet 4 Drive  21.4   6  258.0  110  3.08  3.215  19.44   1   0    3
4  Hornet Sportabout  18.7   8  360.0  175  3.15  3.440  17.02   0   0    3

      carb
0        4
1        4
2        1
3        1
4        2

F-statistic: 39.697515255869035
P-value: 4.978919174400203e-09
Mean mpg differs across cylinder groups

```

Practical No.6**Regression and Its Types**

- A] Implement simple linear regression using a dataset.
B] Explore and interpret the regression model coefficients and goodness-of-fit measures.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
data = pd.read_csv("D:\Rituja\Data.csv")
print("--- Dataset Head ---")
print(data.head())
# 1. Split Data (Input X and Target y)
X = data[['x']]
y = data['y']
# Split into training (80%) and testing (20%)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
# 2. Train Model
model = LinearRegression()
model.fit(X_train, y_train)
# 3. Predict on Test Data
y_pred = model.predict(X_test)
# 4. Evaluation (Aim B)
print("\n--- Model Evaluation ---")
print("Mean Squared Error (MSE): %.2f" % mean_squared_error(y_test, y_pred))
print("R-squared (Accuracy): %.2f" % r2_score(y_test, y_pred))
print("\n--- Model Coefficients ---")
print("Coefficient (Slope):", model.coef_)
print("Intercept:", model.intercept_)
# 5. Predict for a new unknown value (x = 6)
new_data = pd.DataFrame({'x': [6]})
predicted_output = model.predict(new_data)
print("\nPredicted output for x=6:", predicted_output)
```

```
... --- Dataset Head ---
   x  y
0  1  2
1  2  3
2  3  4
3  4  5
4  5  6

--- Model Evaluation ---
Mean Squared Error (MSE): 0.00
R-squared (Accuracy): nan

--- Model Coefficients ---
Coefficient (Slope): [1.]
Intercept: 1.0

Predicted output for x=6: [7.]
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_regression.py:1266: UndefinedMetricWarning
warnings.warn(msg, UndefinedMetricWarning)
```

C] Extend the analysis to multiple linear regression and assess the impact of additional predictors.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
data = pd.read_csv("D:\Rituja\advertising.csv")
print("--- Advertising Dataset ---")
print(data.head())
# 1. Split Data (Multiple inputs now!)
X = data[['Radio', 'TV', 'Newspaper']]
y = data['Sales']
# Split Data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=160)
# 2. Train Model
model = LinearRegression()
model.fit(X_train, y_train)
# 3. Predict and Evaluate
y_pred = model.predict(X_test)
print("\n--- Multiple Regression Evaluation ---")
print("Mean Squared Error: %.2f" % mean_squared_error(y_test, y_pred))
print("R-squared: %.2f" % r2_score(y_test, y_pred))
# 4. Predict for new data
# We must provide all 3 inputs: Radio, TV, Newspaper
new_data = pd.DataFrame({
    'Radio': [8],
    'TV': [20.00],
    'Newspaper': [69.39]
})
predicted_sales = model.predict(new_data)
print("\nPredicted Sales for new input:", predicted_sales)
```

```
... --- Advertising Dataset ---
      TV  Radio  Newspaper  Sales
0  230.1   37.8      69.2   22.1
1   44.5   39.3      45.1   10.4
2   17.2   45.9      69.3   12.0
3  151.5   41.3      58.5   16.5
4  180.8   10.8      58.4   17.9

--- Multiple Regression Evaluation ---
Mean Squared Error: 2.14
R-squared: 0.92

Predicted Sales for new input: [6.64574109]
```

Practical No.7**Logistic Regression and Decision Tree**

A] Build a logistic regression model to predict a binary outcome.

B] Evaluate the model's performance using classification metrics (e.g., accuracy, precision, recall).

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score, classification_report

# 1. Load Data
df = pd.read_csv('/usr/LData.csv') # Ensure "LData.csv" matches your file name
(capital L or D)
X = df[['x']]
y = df['y']

# 2. Split Data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# 3. Train Model
model = LogisticRegression()
model.fit(X_train, y_train)

# 4. Predict
y_pred = model.predict(X_test)

# 5. Accuracy Check
print("Accuracy:", accuracy_score(y_test, y_pred))

# 6. Predict for New Data
X_new = pd.DataFrame({'x': [1.5, 2.8, 4.2]})
y_pred_new = model.predict(X_new)
print("Predicted classes for new data:", y_pred_new)

# 7. Final Metrics (Aim B)
# Note: I added zero_division=0 to prevent errors on small datasets
print("\nModel Performance Metrics:")
print(f"Accuracy: {accuracy_score(y_test, y_pred):.2f}")
print(f"Precision: {precision_score(y_test, y_pred, zero_division=0):.2f}")
print(f"Recall: {recall_score(y_test, y_pred, zero_division=0):.2f}")
print(f"F1-Score: {f1_score(y_test, y_pred, zero_division=0):.2f}")

# Full Report
print("\nClassification Report:\n")
```

```
print(classification_report(y_test, y_pred, target_names=['No Disease', 'Disease'],
zero_division=0))
```

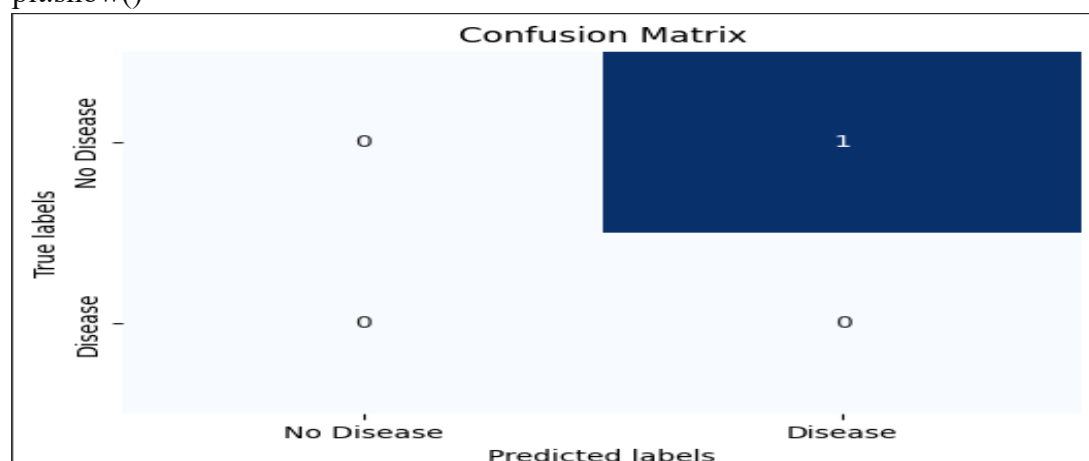
```
Accuracy: 0.0
Predicted classes for new data: [1 1 0]

Model Performance Metrics:
Accuracy: 0.00
Precision: 0.00
Recall: 0.00
F1-Score: 0.00

Classification Report:
```

	precision	recall	f1-score	support
No Disease	0.00	0.00	0.00	1.0
Disease	0.00	0.00	0.00	0.0
accuracy			0.00	1.0
macro avg	0.00	0.00	0.00	1.0
weighted avg	0.00	0.00	0.00	1.0

```
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix
# We use the 'y_test' and 'y_pred' from the code above
# so the chart represents your ACTUAL model, not fake numbers.
cm = confusion_matrix(y_test, y_pred)
# Plot confusion matrix
plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, cmap='Blues', fmt='g', cbar=False,
            xticklabels=['No Disease', 'Disease'],
            yticklabels=['No Disease', 'Disease'])
plt.xlabel('Predicted labels')
plt.ylabel('True labels')
plt.title('Confusion Matrix')
plt.show()
```



C] Construct a decision tree model and interpret the decision rules for classification.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score, classification_report

# 1. Load Data
heart_data = pd.read_csv("/usr/heart.csv")
X = heart_data.drop(columns='target')
y = heart_data['target']

# 2. Split Data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# 3. Construct Model (Aim C Part 1)
model = DecisionTreeClassifier(max_depth=3)
model.fit(X_train, y_train)

# 4. Visualize Tree (Aim C Part 2: "Interpret decision rules")
plt.figure(figsize=(12, 8))
plot_tree(model,
          feature_names=X.columns,
          class_names=['No Disease', 'Disease'],
          filled=True)
plt.title("Decision Tree Visualization")
plt.show()

# 5. Metrics (Evaluation)
y_pred = model.predict(X_test)

print("\nModel Performance Metrics:")
print(f'Accuracy: {accuracy_score(y_test, y_pred):.2f}')
print(f'Precision: {precision_score(y_test, y_pred, zero_division=0):.2f}')
print(f'Recall: {recall_score(y_test, y_pred, zero_division=0):.2f}')
print(f'F1-Score: {f1_score(y_test, y_pred, zero_division=0):.2f}')

print("\nClassification Report:\n")
print(classification_report(y_test, y_pred, target_names=['No Disease',
'Disease'], zero_division=0))
```

Decision Tree Visualization



Model Performance Metrics:

Accuracy: 0.82

Precision: 0.84

Recall: 0.81

F1-Score: 0.83

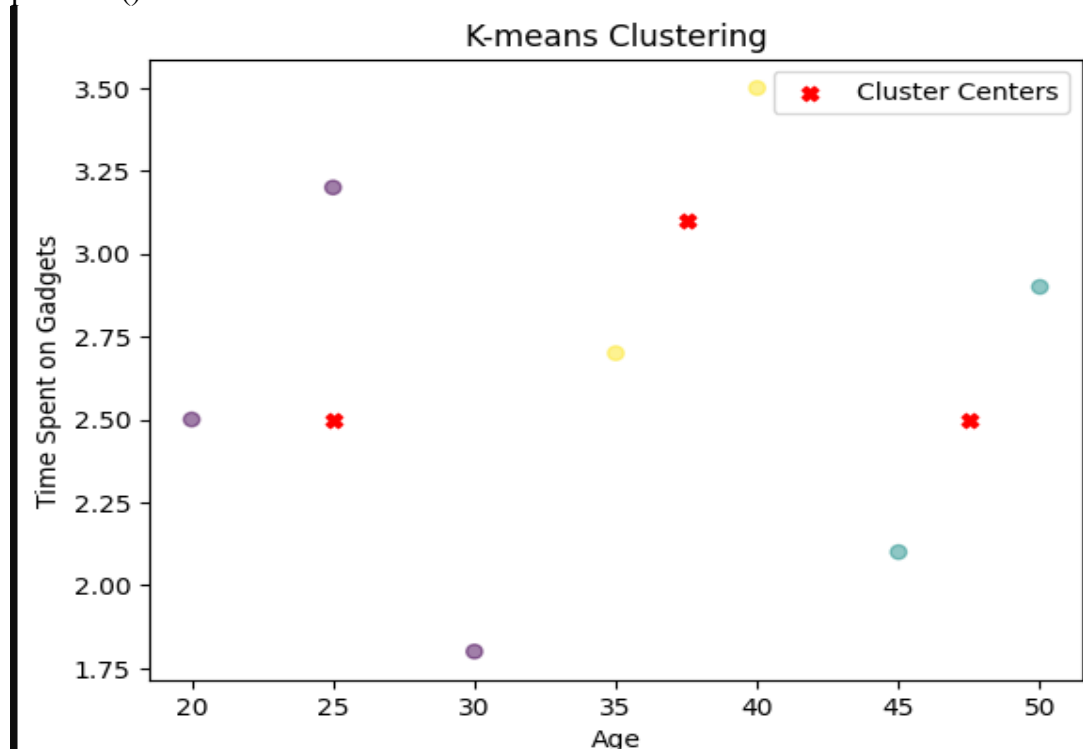
Classification Report:

	precision	recall	f1-score	support
No Disease	0.80	0.83	0.81	29
Disease	0.84	0.81	0.83	32
accuracy			0.82	61
macro avg	0.82	0.82	0.82	61
weighted avg	0.82	0.82	0.82	61

Practical No.8**K-Means Clustering**

- A] Apply the K-Means algorithm to group similar data points into clusters.
- B] Determine the optimal number of clusters using elbow method or silhouette analysis.
- C] Visualize the clustering results and analyse the cluster characteristics.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
# Load the data from the CSV file
df = pd.read_csv("/age.csv")
# Extract features (Age and Time_Spent)
X = df[['Age', 'Time_Spent']]
# Define the number of clusters
k = 3
# Create KMeans instance
kmeans = KMeans(n_clusters=k)
# Fit KMeans to the data
kmeans.fit(X)
# Get cluster centers and labels
centers = kmeans.cluster_centers_
labels = kmeans.labels_
# Plot the data points and cluster centers
plt.scatter(df['Age'], df['Time_Spent'], c=labels, cmap='viridis', alpha=0.5)
plt.scatter(centers[:, 0], centers[:, 1], c='red', marker='X', label='Cluster Centers')
plt.xlabel('Age')
plt.ylabel('Time Spent on Gadgets')
plt.title('K-means Clustering')
plt.legend()
plt.show()
```

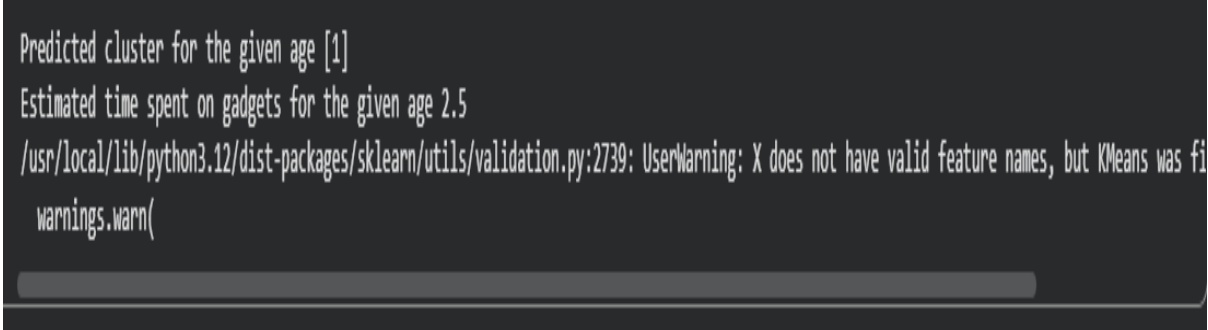


```
# Here, 0 is a placeholder for the time spent (since we want to predict it)
age_unknown = [[55, 0]]

# Use the trained KMeans model to predict the cluster for the given age
# Note: You might get a 'UserWarning' here like in your screenshot. That is normal.
cluster_1 = kmeans.predict(age_unknown)

# Get the cluster center for the predicted cluster
cluster_center_1 = centers[cluster_1]

# Print the predicted cluster and the corresponding cluster center
# (estimated time spent on gadgets)
print("Predicted cluster for the given age", cluster_1)
print("Estimated time spent on gadgets for the given age", cluster_center_1[:, 1][0])
```



```
Predicted cluster for the given age [1]
Estimated time spent on gadgets for the given age 2.5
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but KMeans was fi
warnings.warn(
```

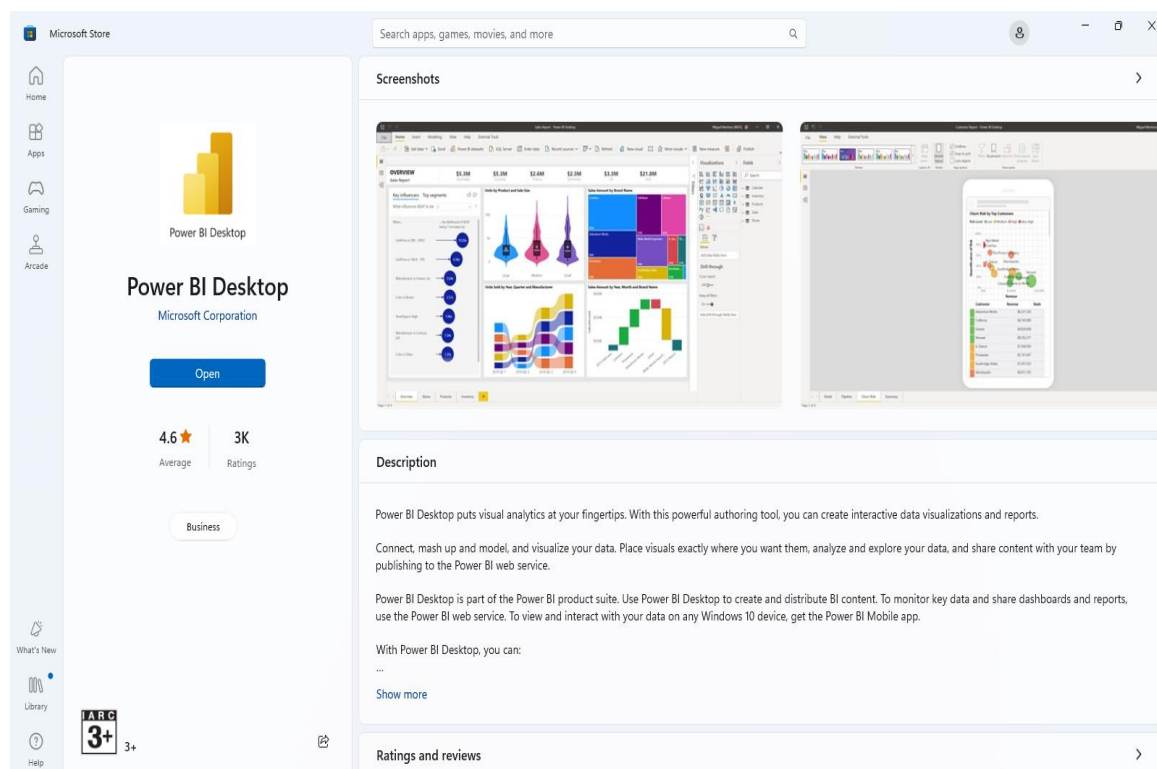
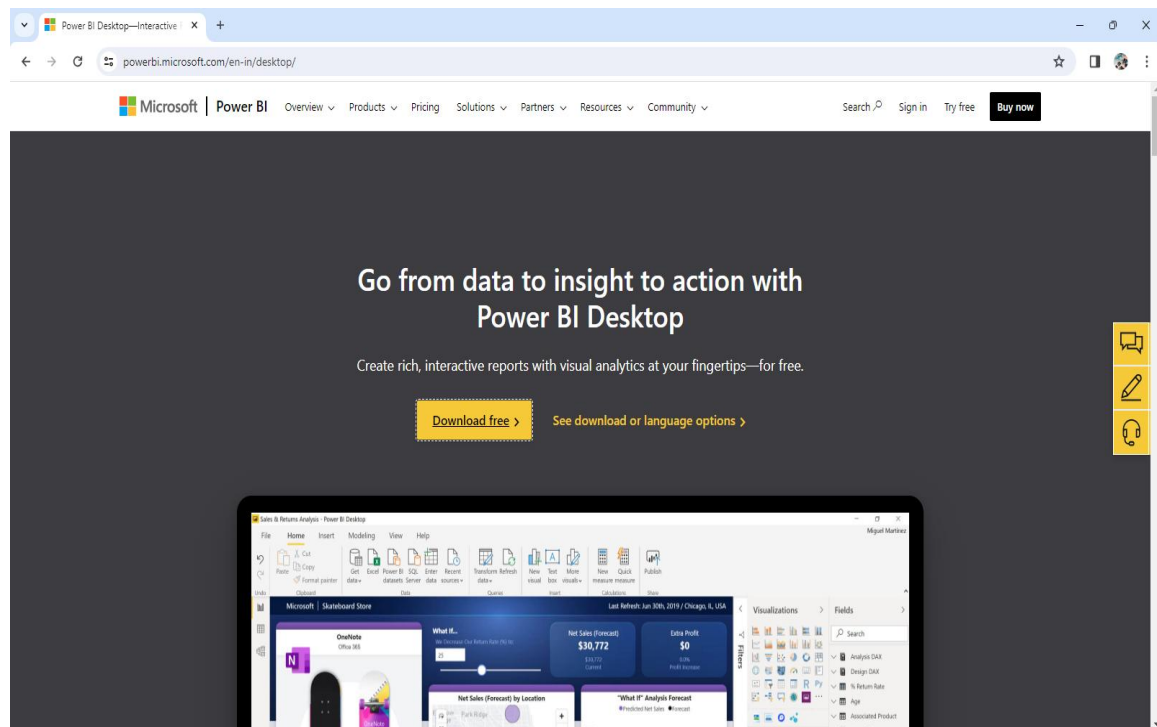
Practical No.9

Data Visualization and Storytelling

A] Create meaningful visualizations using data visualization tools.

Pre-Requirements:

To Install Power BI Desktop follow these steps:



you can also start it with anaconda

COVID-19 Dataset

Number of Confirmed, Death and Recovered cases every day across the globe

MADE WITH PYTHON USES GIT

About Dataset

Context

- A new coronavirus designated 2019-nCoV was first identified in Wuhan, the capital of China's Hubei province
- People developed pneumonia without a clear cause and for which existing vaccines or treatments were not effective.
- The virus has shown evidence of human-to-human transmission

Usability 10.00

License Other (specified in description)

Expected update frequency Daily

Tags Law Coronavirus Public Health Diseases

download 20 mb file

Open

Recent

OneDrive

Find recently opened reports here

Once you've opened some Power BI content, come back to Recents to find it again easily.

Sign in

Options and settings

About

Browse this device

Open

Desktop > DS_April > archive

Name	Date modified	Type	Size
country_wise_latest	05-04-2024 10:27	Comma Separate...	15 KB
covid_19_clean_complete	05-04-2024 10:27	Comma Separate...	3,228 KB
day_wise	05-04-2024 10:27	Comma Separate...	15 KB
full_grouped	05-04-2024 10:27	Comma Separate...	1,814 KB
usa_county_wise	05-04-2024 10:27	Comma Separate...	68,203 KB
worldometer_data	05-04-2024 10:27	Comma Separate...	17 KB

File name: All Files (*.*)

Open Cancel

Visualizations

Build visual

Filters

Values

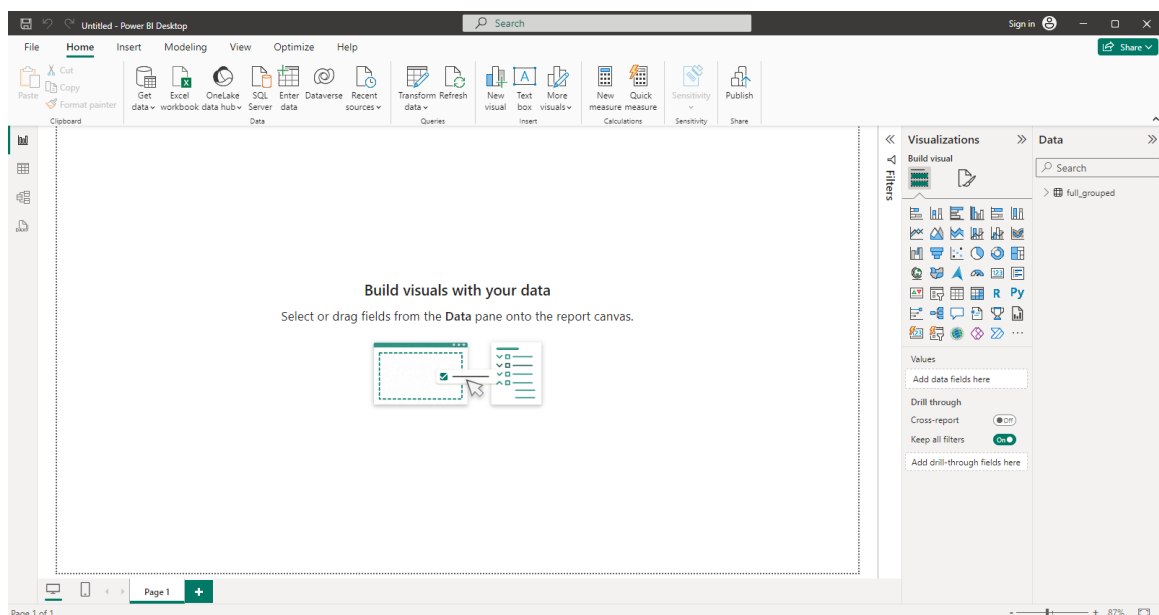
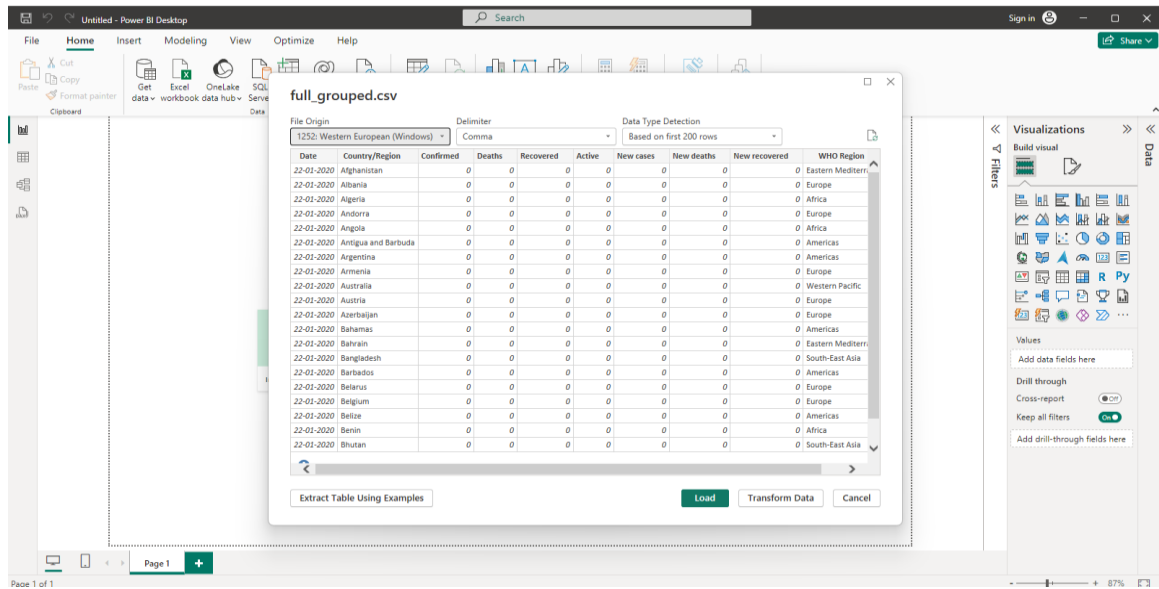
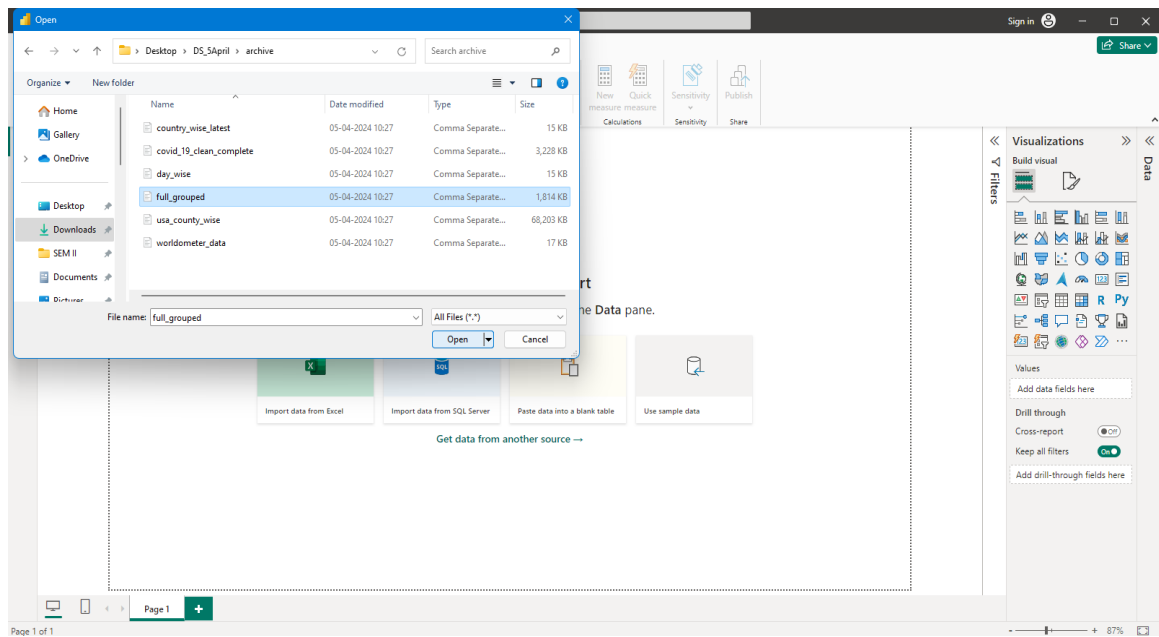
Add data fields here

Drill through

Cross-report

Keep all filters

Add drill-through fields here



File Home Help **Table tools**

Name full_grouped

Structure

- Mark as date table ▾
Calendars
- Manage relationships
Relationships
- New measure
Measure
- Quick measure column
Calculations
- New table
New table

Date	Country/Region	Confirmed	Deaths	Recovered	Active	New cases	New deaths	New recovered	WHO Region
22 January 2020	Lesotho	0	0	0	0	0	0	0	Africa
23 January 2020	Lesotho	0	0	0	0	0	0	0	Africa
24 January 2020	Lesotho	0	0	0	0	0	0	0	Africa
25 January 2020	Lesotho	0	0	0	0	0	0	0	Africa
26 January 2020	Lesotho	0	0	0	0	0	0	0	Africa
27 January 2020	Lesotho	0	0	0	0	0	0	0	Africa
28 January 2020	Lesotho	0	0	0	0	0	0	0	Africa
29 January 2020	Lesotho	0	0	0	0	0	0	0	Africa
30 January 2020	Lesotho	0	0	0	0	0	0	0	Africa
31 January 2020	Lesotho	0	0	0	0	0	0	0	Africa
01 February 2020	Lesotho	0	0	0	0	0	0	0	Africa
02 February 2020	Lesotho	0	0	0	0	0	0	0	Africa
03 February 2020	Lesotho	0	0	0	0	0	0	0	Africa
04 February 2020	Lesotho	0	0	0	0	0	0	0	Africa
05 February 2020	Lesotho	0	0	0	0	0	0	0	Africa
06 February 2020	Lesotho	0	0	0	0	0	0	0	Africa
07 February 2020	Lesotho	0	0	0	0	0	0	0	Africa
08 February 2020	Lesotho	0	0	0	0	0	0	0	Africa
09 February 2020	Lesotho	0	0	0	0	0	0	0	Africa
10 February 2020	Lesotho	0	0	0	0	0	0	0	Africa
11 February 2020	Lesotho	0	0	0	0	0	0	0	Africa
12 February 2020	Lesotho	0	0	0	0	0	0	0	Africa
13 February 2020	Lesotho	0	0	0	0	0	0	0	Africa
14 February 2020	Lesotho	0	0	0	0	0	0	0	Africa
15 February 2020	Lesotho	0	0	0	0	0	0	0	Africa
16 February 2020	Lesotho	0	0	0	0	0	0	0	Africa
17 February 2020	Lesotho	0	0	0	0	0	0	0	Africa
18 February 2020	Lesotho	0	0	0	0	0	0	0	Africa
19 February 2020	Lesotho	0	0	0	0	0	0	0	Africa

Tables: full_grouped (35,156 rows)

Data

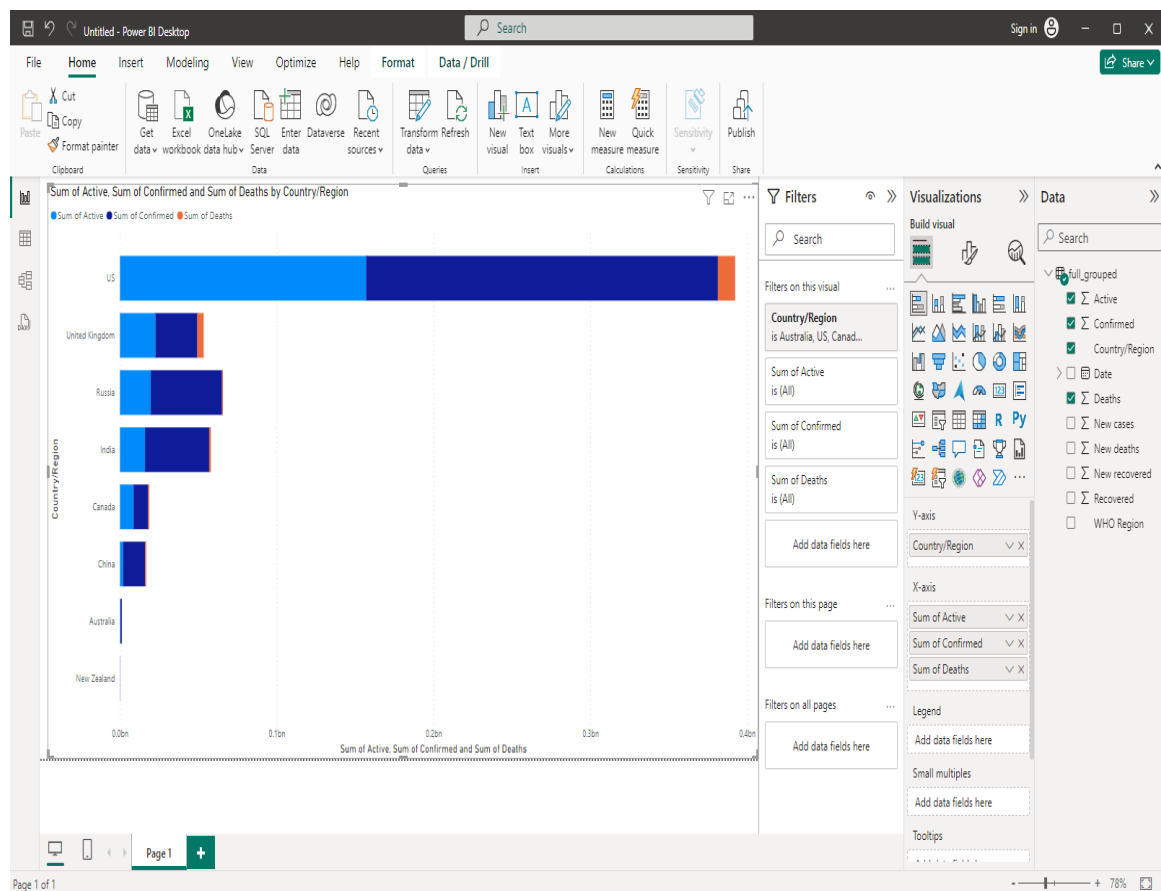
Search

full_grouped

Select country accordingly in filter

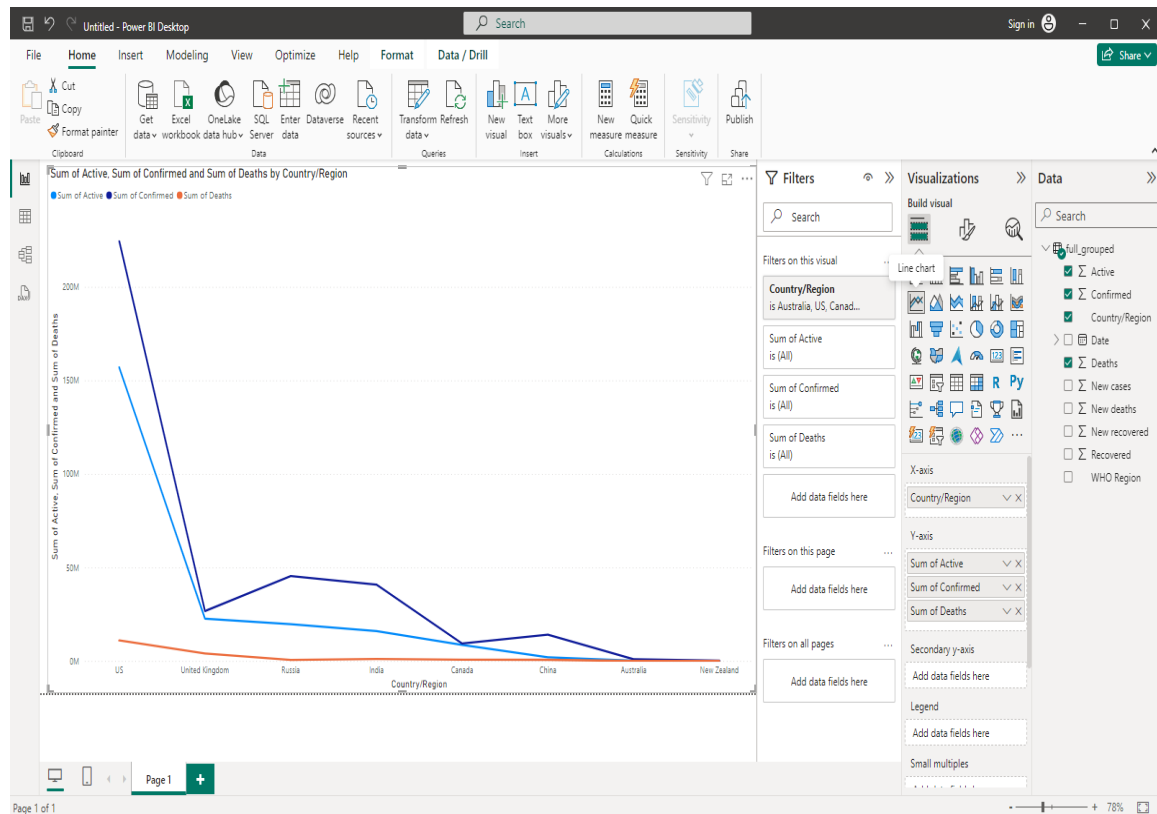
A] Bar chart

Dataset: full_grouped.csv(selected file)



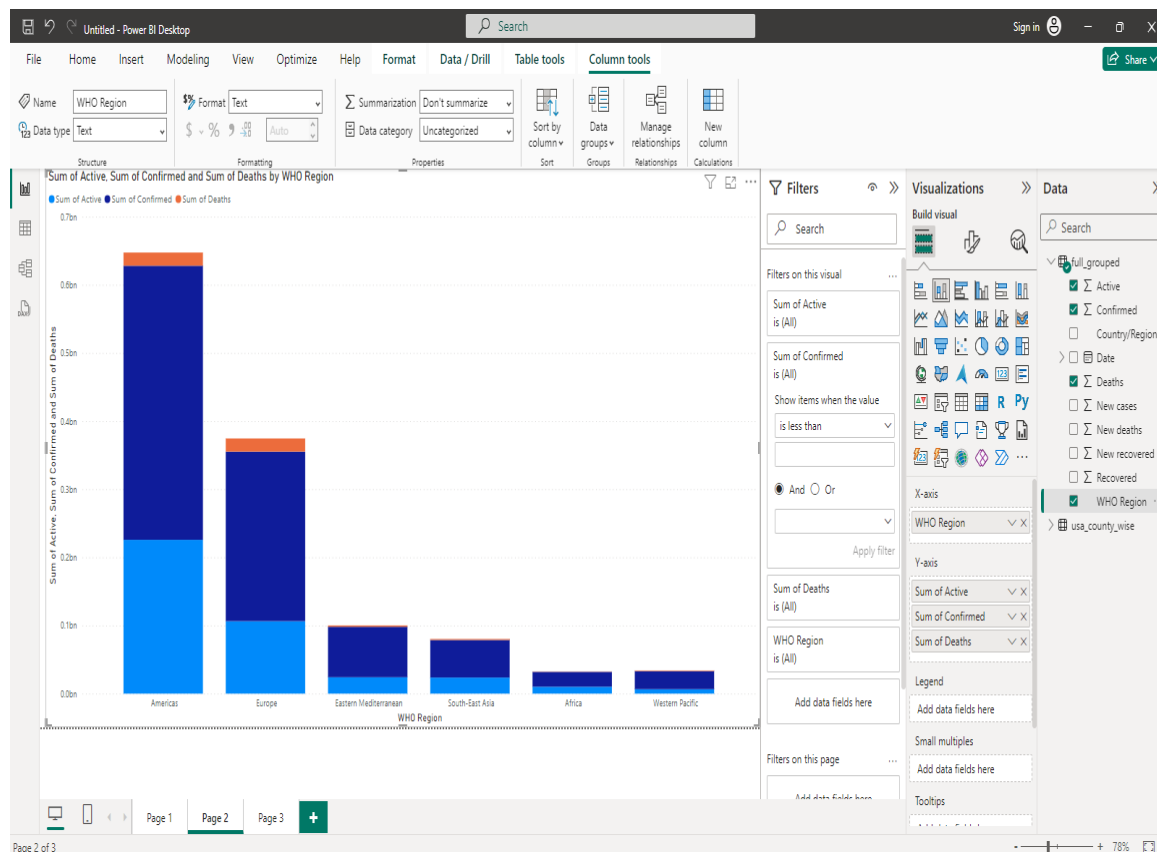
B] Line chart

Dataset: full_grouped.csv



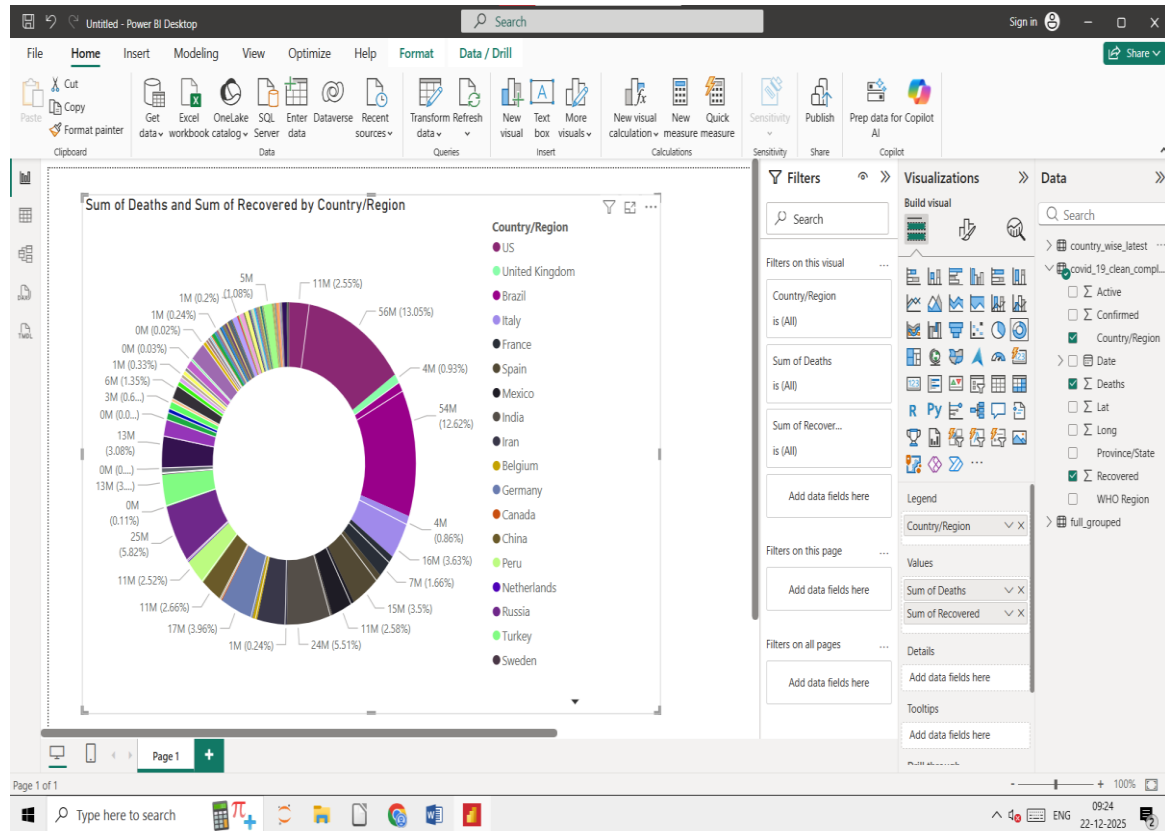
C] Column Chart

Dataset: usa_county_wise.csv



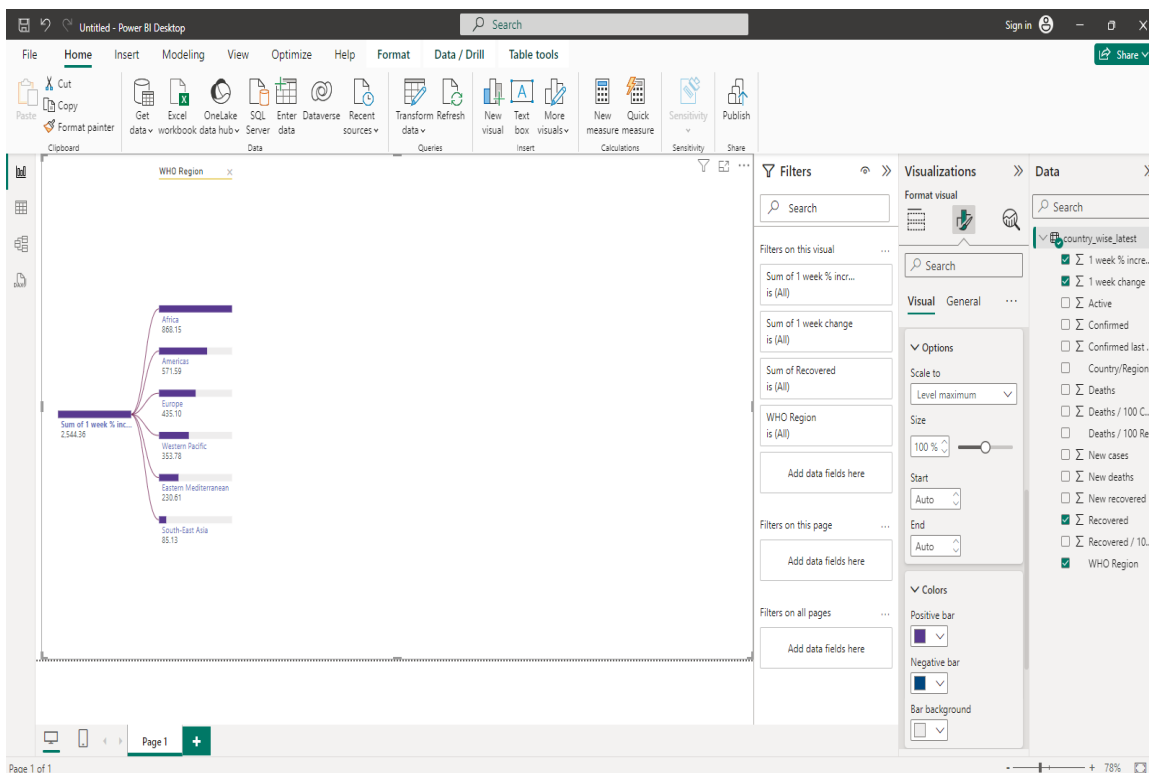
D] Dotnut Chart

Dataset: covid_19_clean_complete.csv(selected file)



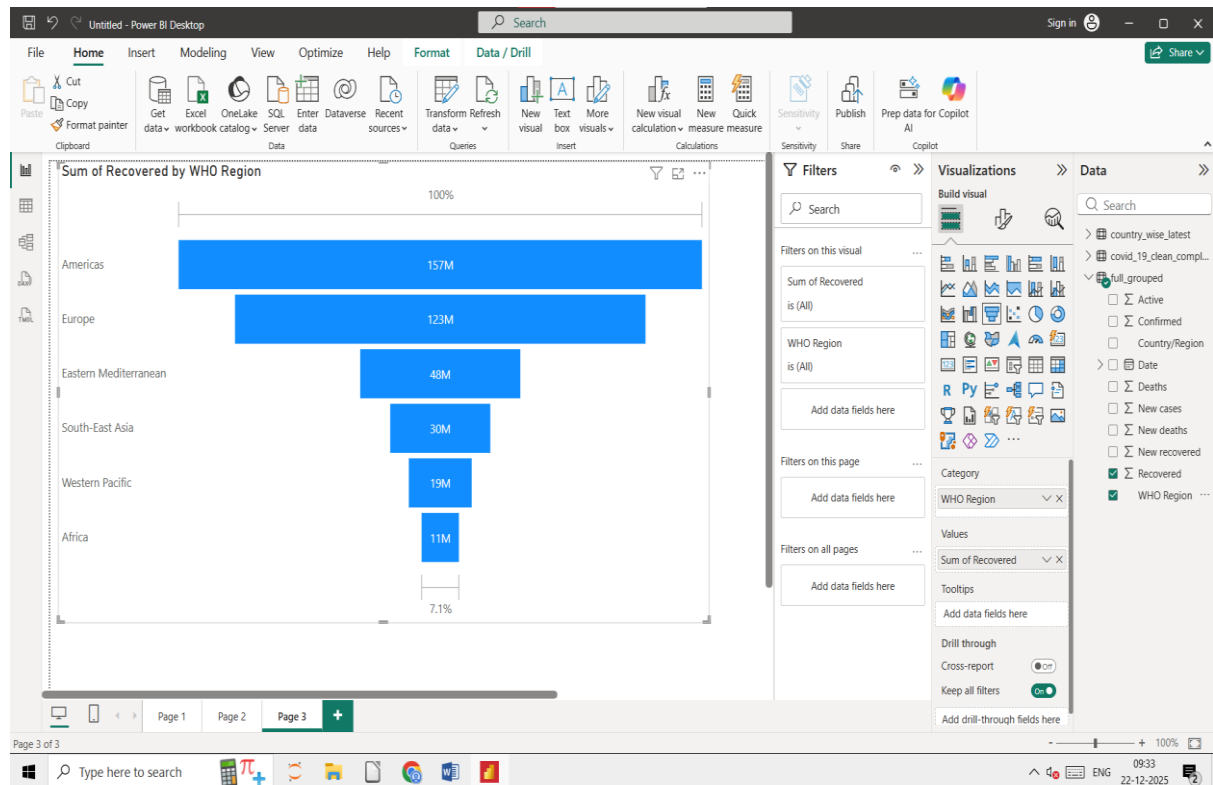
E] Decomposition Tree

Dataset: covid_19_clean_complete.csv



F] Funnel chart

Dataset: full_grouped.csv



G] Scatter chart

Dataset: full_grouped.csv(selected file)

